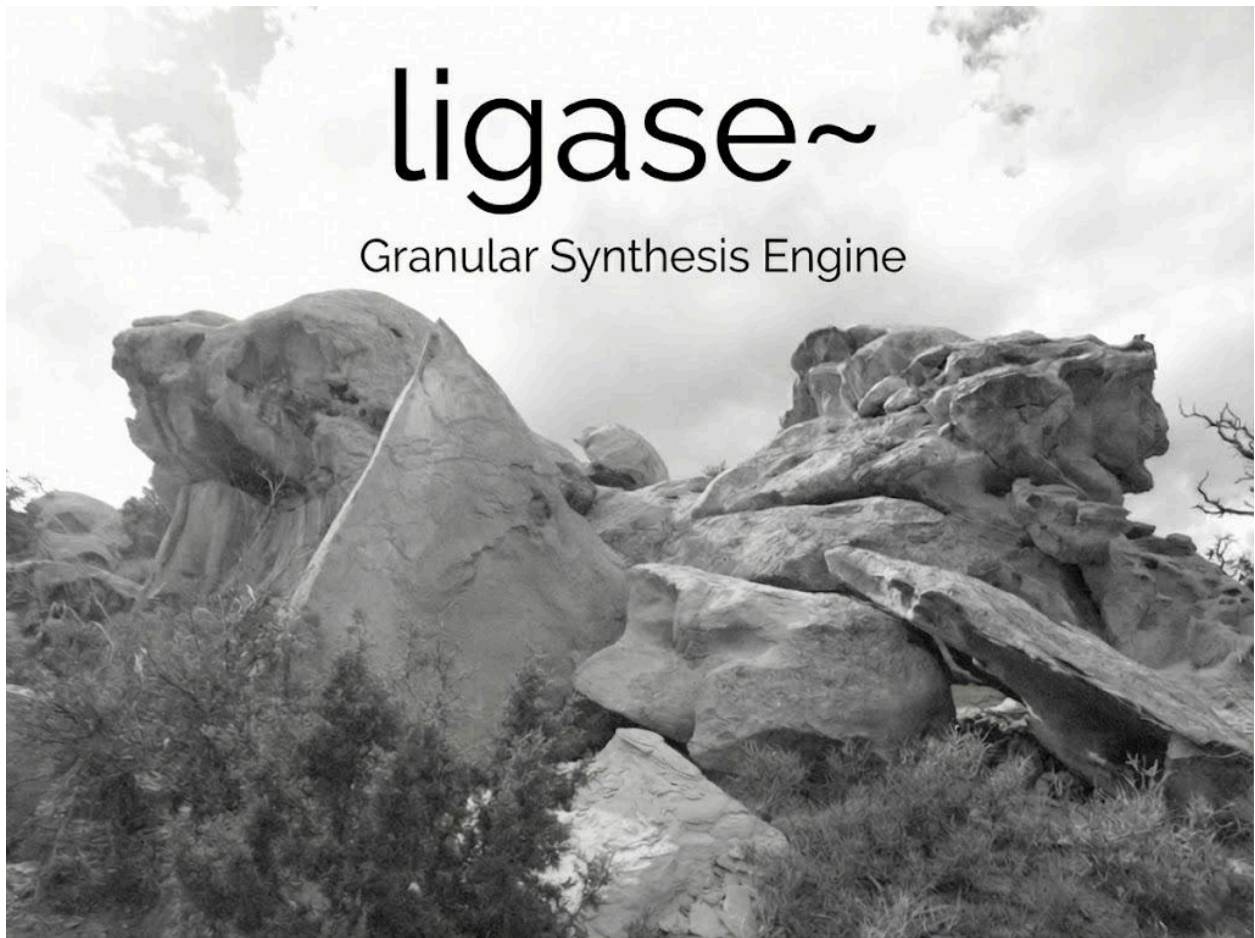




Copyright (C) 2025 Steven Benjamin - Licensed under GNU General Public License

Contents

OVERVIEW.....	2
INLETS & OUTLETS.....	3
MESSAGES.....	4
• Defaults	
PLAYBACK CONTROL.....	9
• Transport	
• Playhead Modes	
• Splice Navigation	
• Splice Creation	
RECORDING CONFIGURATION.....	12
• Record Modes	
• SOS Mode Control	
• SOS Crossfade Parameter	

SPLICE NAVIGATION.....	15
• Direct Navigation • Splice Creation • Splice Removal • Splice Joining	
PLAYHEAD MODES.....	17
GRAIN PARAMETERS.....	19
• Interonset Time • Grain Size • Grain Start Position • Maximum Grains • Amplitude • Pan • Envelope Shape • Nth Grain End Output	
TIMING & QUANTIZATION.....	23
• Clock Management • IOT Quantization • Grain Size Quantization • Delay Time Quantization	
PITCH & SPEED.....	26
DELAY.....	28
DISTORTION.....	29
• Control • Waveshaper Modes • Pre-emphasis / De-emphasis (Inner Filters) • Outer Filters	
MOOG LADDER FILTER.....	33
PARAMETER RANGES & MODULATION.....	34
• Modulation Sources (Generators) • Modulatable Parameters • Messages • N-Body Control • Sampling Timing • Modulation Mapping • Generator Outlet Sends • Parameter Slewing • Generator Reset • Defaults	
QUERY STATE.....	40

OVERVIEW

ligase~ is a granular synthesizer/sampler/looper/delay with real-time recording, reel/splice management, filter, distortion and chaotic parameter modulation inspired by features of the Morphogene, Audiomulch's DLGranulator and Tidalcycles.

ligase~ implements asynchronous granular synthesis with splice-based sample organization. The engine operates on a fixed-size stereo buffer (10 minutes at 48kHz) divided into segments called splices.

Components

Reel: Circular stereo buffer ($\text{MAX_REEL_SECONDS} * 48000$ samples)

Scheduler: Grain pool manager (configurable 1-2000 grains, default 200)

Recorder: Real-time input capture with sound-on-sound mixing

Envelope: Table-based amplitude shaping (parabolic, trapezoidal, cosine)

Grain pool allocation is configurable via ligase.conf file placed in the same directory as the external.

Format: $\text{max_grains} = N$ (range 1-2000).

Grain Generation

1. Trigger timing controlled by IOT (interonset time): 0.0005-2.0 seconds
2. Grain size: 0.001-2.0 seconds (default 0.1)
3. Playhead position determines grain start point within current splice
4. Speed parameter controls playback rate and direction (-4.0 to 4.0)
5. Envelope applied via table lookup with linear interpolation
6. Maximum concurrent grains: 1 to pool_size

Grain Mixing

Constant-power stereo panning (0=left, 0.5=center, 1=right)

Per-grain amplitude scaling: 0.0-2.0 (default 1.0)

Summation of all active grains per sample

No automatic gain compensation

INLETS

1. Audio L - Left input + Messages + Bangs(metro/clock)
 2. Audio R - Right input
 3. GrainSize - Grain duration (0.001-10 seconds)
 4. GrainStart - Normalized start position (0-1)
 5. Speed - Playback rate (-4 to 4)
 6. Organize - Splice selection (0-1)
 7. ScanRate - Scan speed for mode 2 (-1000 to 1000)
 8. SoS - Sound-on-sound mix (0-1)
 9. IOT - Interonset time (0.0005-2 seconds)
 10. MaxGrains - Max concurrent grains (1-pool_size)
 11. GDelay Time - Delay time (0-10 seconds)
 12. GDelay Feedback - Feedback amount (0-1)
 13. GDelay Tone - Lowpass filter (0-1)
 14. GDelay Mix - Dry/wet mix (0-1)
 15. Distortion - Drive intensity (0-1)
 16. Moog Cutoff - Filter cutoff (20-20000 Hz)
 17. Moog Resonance - Filter resonance (0-4)
 18. Moog Mix - Filter dry/wet (0-1)
 19. MIDI Pitch - MIDI note (1-127, 0=inactive)
 20. Envelope Skew - Shape skew (0-1)
 21. Amplitude - Grain amplitude (0-2)
 22. Pan - Stereo position (0-1)
- Unconnected inlets default to stored values (set via messages).

Zero values from unconnected inlets do not overwrite stored parameters.

OUTLETS

1. Signal - Audio output left
2. Signal - Audio output right
3. Bang - Splice end/wrap notification (modes 2 & 3)
4. Bang - Grain onset (every N grains, set via `grain_bang_rate`)

MESSAGES

File Operations

`load <filename>` - Load 48kHz 32-bit stereo WAV
`save <filename>` - Save current reel to WAV

Playback Control

`play <0|1>` - Start/stop playback
`record <0|1>` - Start/stop recording
`recsplice` - Record and create splice on stop
`recinput` - Input-only recording, create splice on stop

Recording Configuration

`sos <0-1>` - Sound-on-sound mix level
`sos_mode <0|1>` - 0=record only, 1=crossfade (default)

Splice Navigation

`shift <delta>` - Move splice selection (integer)
`organize <0-1>` - Select splice by normalized value
`splice` - Add splice marker at current position

Splice Management

`clear_splices` - Remove all markers, clear buffer
`clear_splices_except_current` - Keep only current splice
`clear_current_splice` - Remove current splice
`splice_join_right` - Join current with right neighbor
`splice_join_all` - Join all splices into one

Splice Behavior

splice_create_pos <0|1|2> - 0=playback pos, 1=right, 2=end
splice_jump <0|1> - Jump to new splice on creation
splice_finish_nav <0|1> - Finish playback before nav
splice_split <0|1> - Allow/prevent splitting current

Playhead

playhead <1|2|3> - Static/Scanning/Clock Advance
scanrate <float> - Scan rate for mode 2
speed <-4 to 4> - Playback speed/direction
clock_advance_quant <0|1> - Use current/quantized length
clockstop - Stop clock, keep last BPM

Grain Parameters

grainsize <0.001-10> - Grain duration in seconds
grainstart <0-1> - Position within splice where grains begin (0=start, 1=end)
iot <0.0005-2> - Interonset time in seconds
maxgrains <1-pool_size> - Max concurrent grains
grain_bang_rate <N> - Bang outlet every N grains (0=off)

Envelope

envelope <0|1|2> - Parabolic/Trapezoidal/Cosine
env_skew <0-1> - Envelope shape skew

IOT Quantization

timesig <num>/<denom> - Time signature
quantize <1|2|4|8|16|32|64|128> - Note subdivision
quant <0-1> - Quantization amount

Grain Size Quantization

gs_timesig <num>/<denom>
gs_quantize <subdivision>
gs_quant <0-1>

Delay Quantization

delay_timesig <num>/<denom>
delay_quantize <subdivision>
delay_quant <0-1>

Grain Delay

gdelay_time <0-9.5> - Delay time in seconds

gdelay_feed <0-1> - Feedback amount
gdelay_tone <0-1> - Lowpass filter (0=dark, 1=bright)
gdelay_mix <0-1> - Dry/wet mix
gdelay_clear - Clear delay buffer

Distortion

distortion <0-1> - Drive intensity
distortion_enable <0|1> - Enable/bypass
distortion_oversampling <1|2|4|8> - Oversampling factor
distortion_position <0|1> - Per-grain/Post-mix
dist_waveshaper_mode <0|1|2|3|4> - Tanh/Arctan/Asymmetric/Blend/Polynomial
distortion_pre_hp_freq <30-500> - Pre-HP frequency
distortion_pre_hp_mix <0-1> - Pre-HP mix
distortion_post_lp_freq <2400-10000> - Post-LP frequency
distortion_post_lp_mix <0-1> - Post-LP mix
distortion_notch_freq <Hz> - Notch center frequency
distortion_notch_bw <Hz> - Notch bandwidth
distortion_notch_mix <0-1> - Notch mix
dist_emphasis_mode <0|1> - HP/LP pre-emphasis
dist_emphasis_freq <100-5000> - Emphasis frequency
dist_pregain <0.1-10> - Pre-saturation gain
dist_curve_blend <0-1> - Tanh/arctan morph
dist_drive_pos <1-20> - Positive drive (asymmetric)
dist_drive_neg <1-20> - Negative drive (asymmetric)
dist_poly_c1 <-10 to 10> - Linear coefficient
dist_poly_c2 <-10 to 10> - Quadratic coefficient
dist_poly_c3 <-10 to 10> - Cubic coefficient

Moogladder Filter

moog_cutoff <20-20000> - Cutoff frequency (Hz)
moog_resonance <0-4> - Resonance amount
moog_mix <0-1> - Dry/wet mix
moog_enable <0|1> - Enable/bypass

Parameter Ranging

param_range <param> <min> <max> - Set parameter range
rand_type <type_instance> <param> - Assign generator
param_lock <param1> <param2> <param3>... - Disable modulation and lock parameter(s) at their current modulated value.

Generators

rand_1, rand_2, rand_3, rand_4
perlin_1d_1, perlin_1d_2, perlin_1d_3, perlin_1d_4
perlin_2d_1, perlin_2d_2, perlin_2d_3, perlin_2d_4
lorenz_1, lorenz_2, lorenz_3, lorenz_4
nbody_1, nbody_2, nbody_3, nbody_4

Generator Modulation Outlet Send

modout<N>_source <type> <instance>
modout<N>_range <min> <max>

Noise Frequency

noise_freq <scale> - Set all generator speeds
noise_freq_1|2|3|4 <scale> - generator instance speeds

N-Body Configuration

nbody_mode <instance> <mode> - Set output mode (0-10)
nbody_epsilon <instance> <0.01-1> - Softening parameter
nbody_damping <instance> <0-0.1> - Energy damping
nbody_pump <instance> <0-0.01> - Energy injection
nbody_reset <instance> - Reset to initial conditions

Pitch Control

pitch_mode <0|1|2|3|4> - Off/Semitones/Range/Scale/MIDI
pitch_semitones <-24 to 24> - Fixed transposition
pitch_range <min> <max> - Semitone range
pitch_rand_type <type> - Random generator for pitch
pitch_scale <note1> <note2> ... <noteN> - Scale definition (semitones)

Defaults

Core Parameters

grainsize: 0.1 seconds
grainstart: 0.5 (center of splice)
speed: 1.0 (normal forward)
iot: 0.1 seconds
maxgrains: 4
amplitude: 1.0
pan: 0.5 (center)
env_skew: 0.5 (symmetric)

Envelope

type: ENVELOPE_COSINE
table_length: 4096 samples

Playhead

mode: PLAYHEAD_MODE_STATIC (mode 1)
scanrate: 1.0

Recording

sos: 0.5 (50/50 mix)
sos_mode: 1 (Morphagene-style crossfade)
mode: RECORD_MODE_OVERDUB

Splice

create_pos: 0 (at playback position)
jump: 0 (stay in current)
finish_nav: 0 (immediate)
split: 0 (allow split)

Effects

gdelay_time: 0.0 (off)
gdelay_feed: 0.0
gdelay_tone: 0.5
gdelay_mix: 0.0
distortion: 0.0 (clean)
distortion_enable: 1
distortion_oversampling: 4x
distortion_position: 1 (post-mix)
moog_cutoff: ~1000 Hz
moog_resonance: 0.0
moog_mix: 0.0
moog_enable: 1

Distortion Filters

pre_hp_freq: 30 Hz
pre_hp_mix: 0.5
post_lp_freq: 16000 Hz
post_lp_mix: 0.5
notch_freq: 3000 Hz
notch_bandwidth: 500 Hz
notch_mix: 0.0 (inactive)
emphasis_freq: 800 Hz
emphasis_mode: 0, EMPHASIS_MODE_HP
pregain: 1.0

Pitch Control

mode: PITCH_MODE_OFF
semitones: 0.0
midi_note: 60 (C4)

Quantization

All quant_amount: 0.0 (off)
timesig: 4/4
subdivision: 4 (quarter note)

Noise Generators

All frequency_scale: 1.0 (normal)
All rand_type: RAND_TYPE_RAND
All rand_instance: 0 (generator 1)

Pool

pool_size: 200 grains (configurable via ligase.conf)

Buffer

Sample rate: 48000 Hz (fixed)
Max length: 600 seconds
Channels: 2 (stereo)

PLAYBACK CONTROL

Transport

play <float> Start/stop grain triggering. 0 stops, non-zero starts.

When started:

- Checks reel length > 0. Posts error and aborts if empty.
- Initializes playback_position to current splice start.
- Sets is_playing and is_triggering flags.

When stopped:

- Clears is_playing and is_triggering.
- Active grains finish their envelopes.

Default: 0 (stopped)

record <float> Overdub recording. Argument: 0 stops, non-zero starts.

Start: Sets RECORD_MODE_OVERDUB, begins recording.

Stop: If mode was NEW_SPLICE or INPUT_ONLY, creates splice at recorded segment boundary.

recsplice <band> Record with splice creation on stop.

- Sets RECORD_MODE_NEW_SPLICE
- Posts starting position (current reel length)
- Creates splice when recording stops

recinput Record input only, no sound-on-sound mixing.

- Sets RECORD_MODE_INPUT_ONLY
- Ignores sos parameter during recording
- Creates splice when stopped

Playhead Modes

playhead 1|2|3 Set playhead behavior.

Mode 1: Static (default)

Playback position fixed. grain_start parameter (inlet 4) sets grain spawn location directly
Grain position: $\text{splice_start} + (\text{grain_start} \times \text{splice_length})$

Detects grain_start parameter wraps (change > 0.5), bangs outlet 3. No playback_position advancement.

Mode 2: Scanning

Playback position advances continuously at scan_rate (samples per sample).

Grain position: $\text{playback_position} + (\text{grain_start} \times \text{splice_length})$

Advancement: $\text{playback_position} += \text{scan_rate}$ every sample

Wraps at splice boundaries, bangs outlet 3 on wrap. Checks pending splice navigation after wrap.

Negative scan_rate = reverse.

Mode 3: Clock Advance

Playback position advances only when clock bang received (inlet 1).

Advance amount: $\text{grain length} \times \text{sample_rate}$

Grain length source: current or quantized (see `clock_advance_quant`)

Grain position: $\text{playback_position} + (\text{grain_start} \times \text{splice_length})$ between bangs Wraps, bangs outlet, checks pending navigation like Mode 2.

`scanrate <float>` Set scanning speed for Mode 2.

1.0 = real-time forward

-1.0 = real-time reverse

0.0 = static

Default: 1.0

`clock_advance_quant 0|1` Select grain length source for Mode 3 advance.

0 = current actual grain length (varies per grain)

1 = quantized grid length (requires valid BPM and `gs_quant_grid_ms > 0`)

Default: 0

Splice Structure

- Array of position markers (0-300 maximum)
- Each splice defined by start marker, extends to next marker or buffer end
- Current splice selected via `organize` parameter (0.0-1.0 maps to splice index)
- Shift message: relative navigation (shift 1 = next, shift -1 = previous)

Splice Bounds

Grains read exclusively within current splice boundaries. Position wraps at `splice_end` back to `splice_start`. This creates independent loop regions within the buffer.

Splice Navigation

`organize <float>` Navigate to splice by normalized position (0.0-1.0).

Maps to index: $\text{int}(\text{value} \times (\text{splice_count} - 1))$

Clamps input to [0.0, 1.0]

If `splice_finish_nav = 0`: immediate navigation, resets `playback_position` to splice start

If `splice_finish_nav = 1`: queues navigation, executes after current splice wraps

`shift <int>` Move splice selection by delta.

Adds delta to `current_splice` index.

Clamps to [0, count-1]. No wraparound.

Examples: shift 1 = next, shift -1 = previous

If splice_finish_nav = 1: queues as pending

If splice_finish_nav = 0: immediate

Splice Creation

splice Create splice marker.

Position determined by splice_create_pos:

- 0: at playback_position (if 0 and reel exists, uses reel_length)
- 1: at current splice end boundary
- 2: at reel end

Rate limited: minimum 10ms between creation

Posts error if MAX_SPLICES (64) reached

splice_create_pos 0|1|2 Set splice creation positionmode.

0 = at playback position (default)

1 = right of current splice boundary

2 = at end of reel (Morphogene append)

splice_jump 0|1 Control navigation after creating new splice.

0 = stay in current splice (default)

1 = jump to newly created splice immediately

splice_finish_nav 0|1 Control navigation timing for organize and shift messages.

0 = immediate navigation (default)

1 = queue navigation, execute after current splice wraps

splice_split 0|1 Control splice splitting behavior.

0 = allow splitting current splice (default)

1 = preserve current splice length

RECORDING CONFIGURATION

Record Modes

Three recording modes, set by message commands. Mode determines recording behavior and splice creation.

record <float> Overdub mode. Argument: 0 stops, non-zero starts.

- Sets RECORD_MODE_OVERDUB
- Start: positions record_position at current_splice_start
- Records into current splice bounds
- Continues past splice boundary into adjacent splice (cross-splice recording)
- Mixing controlled by crossfade_mix (sos value)
- Stop: Posts "recording stopped" only

recsplice New splice mode.

- Sets RECORD_MODE_NEW_SPLICE
- Start: positions record_position at reel length (append)
- Stores new_splice_start = current reel length
- Mixing controlled by crossfade_mix (sos value)
- Stop: Creates splice marker at new_splice_start position, sets as current splice
- Posts splice creation confirmation with position

recinput Input-only mode.

- Sets RECORD_MODE_INPUT_ONLY
- Start: positions record_position at reel length (append)
- Stores new_splice_start = current reel length
- Records input directly, ignores sos/crossfade_mix completely
- Stop: Creates splice marker at new_splice_start position, sets as current splice
- Posts splice creation confirmation with position

Default mode: RECORD_MODE_OVERDUB

SOS Mode Control

sos_mode 0|1 Set Sound-on-Sound operational mode.

Mode 0: Record Only

- Output: Constant-power crossfade between input and granular
 - sos = 0.0: 100% granular (input_gain = 0.0, granular_gain = 1.0)
 - sos = 0.5: Equal power mix (both ≈ 0.707)
 - sos = 1.0: 100% input (input_gain = 1.0, granular_gain = 0.0)
 - Formula: $\text{out} = \text{input} \times \sin(\text{sos} \times \pi/2) + \text{granular} \times \cos(\text{sos} \times \pi/2)$

- Recording: happens before output mixing
 - Records raw input only (not the mixed output)
 - crossfade_mix forced to 1.0 (full replacement, no sound-on-sound)
 - sos parameter does NOT affect recording in this mode
 - Always replaces buffer content with input signal
- Output and recording are decoupled:
 - Recording: Raw input → buffer (replace mode)
 - Output: Input + granular constant-power mix (monitoring)

Mode 1: Morphogene (default)

- Output: Constant-power crossfade between input and granular
 - $\text{output} = \text{input} \times \sin(\text{sos} \times \pi/2) + \text{granular} \times \cos(\text{sos} \times \pi/2)$
 - sos = 0.0: 100% granular (input_gain = 0.0, granular_gain = 1.0)
 - sos = 0.5: Equal power mix (both ≈ 0.707)
 - sos = 1.0: 100% input (input_gain = 1.0, granular_gain = 0.0)
 - sos from signal inlet (inlet 8) or stored sos_value
 - Uses inlet value if non-zero, else uses stored value
 - Clamped to [0.0, 1.0]
 - NaN/Inf protection via isfinite() check
- Recording: captures output signal
 - What you hear is what gets recorded
 - Forces crossfade_mix = 1.0 during recording (replace mode)
 - Restores original crossfade_mix after recording
- When no playback (reel length = 0):
 - Output = input × sos (linear VCA behavior)
 - Records scaled input

Default: 1 (Morphogene mode)

SOS Crossfade Parameter

sos <float> Set crossfade mix value.

Range: 0.0 to 1.0 (clamped in processing)

Default: 0.5

SPLICE NAVIGATION

Direct Navigation

shift <int> Navigate by relative offset.

Adds delta to current_splice index. Clamps to [0, count-1]. No wraparound at boundaries.

- Positive: move right through array
- Negative: move left through array
- 0 stops at first splice
- count-1 stops at last splice

If finish_before_nav = 1: calculates target with wraparound $((\text{target} \% \text{count}) + \text{count}) \% \text{count}$, stores as pending, executes after playback wraps at splice boundary.

If finish_before_nav = 0: immediate navigation.

organize <float> Navigate by normalized position.

Maps input [0.0-1.0] to discrete splice index: $\text{int}(\text{value} \times (\text{count} - 1))$

Clamps input to [0.0, 1.0] before mapping.

Examples: 0.0 = first splice, 0.5 = middle splice, 1.0 = last splice

If finish_before_nav = 1: calculates target, stores as pending.

If finish_before_nav = 0: immediate navigation.

Splice Creation

splice Create splice marker.

Position determined by create_position setting:

Mode 0 (default): At playback_position

- If playback_position = 0 and reel exists, uses reel_length instead
- If split_current = 1 and position would split current splice (between splice_start and splice_end), moves to splice_end

Mode 1: At current splice end boundary (splice_end)

Mode 2: At reel end (reel_length)

Rate limited: minimum 10ms between creation. Posts error if violated.

Maximum: 64 splices (MAX_SPLICES). Posts error if exceeded.

After creation, if `jump_to_new = 1`: sets `current_splice` to new index, resets `playback_position` to new splice start.

`splice_create_pos 0|1|2` Set creation position mode. 0 = at playback position

1 = right of current splice

2 = at end of reel

Default: 0

`splice_jump 0|1` Control navigation after splice creation.

0 = stay in current splice (default)

1 = jump to newly created splice, reset `playback_position`

`splice_finish_nav 0|1` Control navigation timing for shift and organize.

0 = immediate navigation (default)

1 = queue navigation, execute when playback wraps at splice boundary

Pending navigation stored in `pending_splice`. Cleared after execution.

`splice_split 0|1` Control current splice splitting.

0 = allow splitting current splice (default)

1 = preserve current splice length

When enabled (mode 1), if `create_position = 0` and calculated position falls within current splice bounds, position moves to `splice_end`.

Splice Removal

`clear_splices` Remove all splice markers, clear entire audio buffer.

Calls `splice_clear()` and `reel_clear()`. Destructive operation. Sets `count = 0`, `current_splice = 0`.

`clear_splices_except_current` Remove all splices except current, clear buffer except current splice audio.

Destructive. Extracts current splice bounds, clears buffer outside bounds, reorganizes markers:

- If current has right neighbor: creates two markers (position 0 and `splice_length`)
- If current extends to reel end: creates one marker (position 0)

Current splice moves to position 0. Sets `current_splice = 0`.

`clear_current_splice` Remove current splice marker immediately.

Uses `splice_remove()` to delete marker at `current_splice` index. Shifts subsequent markers down. Active grains finish naturally.

Adjusts current_splice index if needed:

- If count becomes 0: current_splice = 0
- If current_splice >= count: current_splice = count - 1

Posts count of remaining splices.

Splice Joining

`splice_join_right` Join current splice with right neighbor.

Calculates right neighbor with wraparound: $(\text{current_splice} + 1) \% \text{count}$

Removes marker at right neighbor (boundary between them). Updates current_splice index if $\text{right_splice} \leq \text{current_splice}$.

No effect if only one splice exists.

`splice_join_all` Join all splices into one continuous splice. Calls `splice_clear()`. Entire reel becomes single splice with no markers. Sets count = 0, current_splice = 0.

Posts number of splices joined.

PLAYHEAD MODES

Three modes control how playback position advances and where grains spawn within the current splice.

`playhead 1|2|3` Set playhead mode.

Mode 1: Static (default)

Playback position fixed. `grain_start` signal (inlet 4) directly controls grain spawn location.

Grain position: $\text{splice_start} + (\text{grain_start} \times \text{splice_length})$

grain_start range: 0.0 to 1.0

0.0 = splice start, 0.5 = splice middle, 1.0 = splice end

Wrap detection: When grain_start change exceeds 0.5 (indicating wrap from 1.0→0.0 or 0.0→1.0), bangs outlet 3.

playback_position: Never advances. Remains at value set by play message (splice start).

grain_start tracking: Stores prev_grain_start each block for wrap detection.

Mode 2: Scanning

Playback position advances continuously. grain_start offsets grain spawn relative to moving playhead.

Grain position: $\text{playback_position} + (\text{grain_start} \times \text{splice_length})$ Wraps to stay within splice bounds.

Advancement: $\text{playback_position} += \text{scan_rate}$ every audio sample

scan_rate units: samples per sample

1.0 = real-time forward

1.0- = real-time reverse

0.0 = static (no advancement)

2.0 = double-speed forward

Boundary wrapping:

When $\text{playback_position} \geq \text{splice_end}$: wraps by subtracting splice_length

When $\text{playback_position} < \text{splice_start}$: wraps by adding splice_length

Bangs outlet 3 on wrap. Checks pending_splice navigation after wrap, executes if queued.

Mode 3: Clock Advance

Playback position advances only when clock bang received (inlet 1). Grains spawn at static position between bangs.

Grain position: $\text{playback_position} + (\text{grain_start} \times \text{splice_length})$ Wraps to stay within splice bounds.

Advancement trigger: Bang to inlet 1 sets clock_bang_received flag.

Advance amount: $\text{grain_length} \times \text{sample_rate}$ (converted from seconds to samples)

Grain length source:

Determined by clock_advance_quant setting and current state.

If $\text{clock_advance_quant} = 0$: uses quantized_grain_size (current grain length from perform cycle)

If $\text{clock_advance_quant} = 1$ AND $\text{gs_quant_grid_ms} > 0$ AND $\text{bpm} > 1.0$: Uses $\text{gs_quant_grid_ms} / 1000.0$ (quantized grid length in seconds)

Otherwise: uses quantized_grain_size

Per-bang behavior:

Flag checked once per perform cycle. If set, advances playback_position, wraps at boundaries, bangs outlet 3 if wrapped, checks pending navigation, then clears flag.

Grains trigger normally at IOT rate at current playback_position while flag is clear.

Control Messages

scanrate <float> Set scan rate for Mode 2.

Applied per audio sample as: $\text{playback_position} += \text{scan_rate}$

Default: 1.0

clock_advance_quant 0|1 Select grain length source for Mode 3 advancement.

0 = use current grain length (varies per grain)

1 = use quantized grid length (requires valid BPM and **gs_quant_grid_ms**)

Default: 0

GRAIN PARAMETERS

Interonset Time

iot <float> Set time interval between grain starts.

Range: 0.0005 to 2.0 seconds

Clamped to bounds.

Converted to **grain_trigger_period**: $\text{int}(\text{iot} \times \text{sample_rate})$ samples

When $\text{iot} < \text{grain_size}$: grains overlap

When $\text{iot} > \text{grain_size}$: grains separate

When $\text{iot} = \text{grain_size}$: continuous stream

Signal inlet 8: updates if value in range [0.001, 2.0], else keeps current value.

Quantization: If **quant_amount** > 0 and BPM valid:

$\text{grain_trigger_period} = \text{base_period} \times (1 - \text{quant_amount}) + \text{quant_period} \times \text{quant_amount}$

Minimum **grain_trigger_period** enforced: 1 sample (prevents CPU overload)

Default: 0.1 seconds (100ms)

Grain Size

grainsize <float> Set grain duration.

Range: 0.001 to 10.0 seconds

Clamped to bounds.

Signal inlet 2: updates if value in range [0.001, 10.0], else keeps current value.

Default: 0.1 seconds (100ms)

Grain Start Position

grainstart <float> Set normalized grain spawn position within splice.

Range: 0.0 to 1.0

0.0 = splice start, 0.5 = splice middle, 1.0 = splice end

Signal inlet 4: updates if value in range [0.0, 1.0]

Special handling: if inlet reads 0.0 and stored value is non-zero, assumes inlet unconnected and preserves stored value.

Interacts with playhead mode:

- Mode 1 (Static): $\text{grain_pos} = \text{splice_start} + (\text{grainstart} \times \text{splice_length})$
- Mode 2 (Scanning): $\text{grain_pos} = \text{playback_position} + (\text{grainstart} \times \text{splice_length})$
- Mode 3 (Clock): $\text{grain_pos} = \text{playback_position} + (\text{grainstart} \times \text{splice_length})$

Default: 0.5 (middle)

Maximum Grains

maxgrains <int> Set maximum concurrent active grains. Range: 1 to pool_size

pool_size read from ligase.conf (default: 200)

Clamped to [1, pool_size]

Controls polyphony limit. Grain scheduler uses free-list allocation. When limit reached, oldest grains replaced.

Signal inlet 9: updates if value in range [1, pool_size], else keeps current value.

Default: 4

Amplitude

amplitude <float> Set grain output level.

Range: 0.0 to 2.0

Clamped to bounds.

Applied during grain triggering. Passed to scheduler_trigger_grain().

Signal inlet 19: amplitude_in (read from perform cycle)

Default: 0.75

Per-grain amplitude scaling and randomization

There are two operational modes based on param_range configuration:

Mode 1: Direct Control (param_range disabled, default)

- Uses amplitude value directly from inlet or message

- No per-grain randomization
- Consistent amplitude across all grains

Mode 2: Modulated Range (param_range enabled)

- Samples amplitude from [min, max] range per grain
- Uses inlet/message value as base for sampling
- Random source determines variation pattern
- Examples:

`param_range amplitude 0.5 1.0` → grains vary 0.5-1.0

`rand_type perlin_1d_1 amplitude` → smooth amplitude evolution

`rand_type rand_1 amplitude` → random per-grain levels

Pan Mode

`pan_mode <int>` Select panning algorithm.

Mode 0: Constant-Power Mono Panning (default)

Sums stereo source to mono before panning

Each grain becomes a point source in the stereo field

Ideal for spatial positioning and grain cloud diffusion

Mode 1: Stereo Balance

Preserves stereo width of source material

Applies constant-power gains independently to L/R channels

Maintains stereo image character while adjusting balance

Ideal for stereo sources where width should be preserved

Default: 0 (constant-power mono panning)

Pan Law

Both modes use constant-power panning to maintain consistent perceived loudness:

$$\text{pan_angle} = \text{pan} * (\pi/2)$$

$$\text{left_gain} = \cos(\text{pan_angle}) \quad // \quad 1.0 \rightarrow 0.707 \rightarrow 0.0$$

$$\text{right_gain} = \sin(\text{pan_angle}) \quad // \quad 0.0 \rightarrow 0.707 \rightarrow 1.0$$

This ensures $\cos^2(\theta) + \sin^2(\theta) = 1$ at all pan positions, preventing the "hole in the middle" effect where center-panned signals sound quieter.

At center (pan=0.5): Both channels at 0.707 (-3dB) = 0dB total power

Pan Modulation

Pan can be modulated per-grain using param_range:

Direct Control (param_range disabled)

Uses pan value directly from inlet or message

No per-grain randomization

All grains at same stereo position

Default behavior

Modulated Range (param_range enabled):

Samples pan position from [min, max] range per grain

Uses inlet/message value as base for sampling

Random source determines spatial pattern

Creates spatial diffusion effects

Examples:

param_range pan 0.0 1.0

rand_type perlin_1d_1 pan ← smooth stereo movement

param_range pan 0.3 0.7

rand_type rand_1 pan ← random center-focused placement

param_range pan 0.0 0.5

rand_type lorenz_1 pan ← chaotic left-side positioning

Spatial Cloud Effects:

Wide range + fast modulation = diffuse stereo cloud

Narrow range + slow modulation = subtle stereo shimmer

Per-grain randomization creates spatial depth impossible with traditional panning

Envelope Shape

envelope 0|1|2 Set grain envelope type.

Type 0: Parabolic (Welch window)

Formula: $1 - 4(x - 0.5)^2$

Smooth, gentle envelope. Quadratic curve.

Type 1: Trapezoidal

Linear attack (10% default), sustain, linear release (10% default). Hard attack/release with flat sustain. Percentages adjusted by skew.

Type 2: Cosine (Hann window, default)

Formula: $0.5 \times (1 - \cos(2\pi \times x))$

Smoothest envelope. Reduces spectral artifacts.

Envelope table length: 4096 samples

Preserves skew value when type changes.

Default: ENVELOPE_COSINE (type 2)

Envelope Skew

Skew remaps time axis before envelope calculation, shifting peak position while maintaining smoothness.

`env_skew <float>` Set envelope attack/decay balance.

Range: 0.0 to 1.0

Clamped to bounds.

0.5 = symmetric (peak at center)

< 0.5 = shorter attack, longer decay (peak shifts left)

>0.5 = longer attack, shorter decay (peak shifts right)

Regenerates envelope table immediately on change.

Signal inlet 18: updates if value in range [0.001, 1.0] and different from current, else keeps current value.

Grain Bang Output

`grain_bang_rate <int>` Set grain onset bang rate.

Range: 0 to 1000

0 = disabled (no bangs)

1 = bang every grain

2 = bang every 2nd grain

N = bang every Nth grain

Bangs outlet 4 when grain_bang_counter reaches grain_bang_rate. Counter resets on rate change.

TIMING & QUANTIZATION

Clock Management

Clock bangs to inlet 1 establish tempo. BPM calculated from the interval between bangs.

Bang to inlet 1: Calculate BPM and update quantization grids.

First bang: stores timestamp, no BPM calculation

Second bang onward: calculates interval, derives BPM

BPM calculation: $60000.0 / \text{interval_ms}$

Sets clock_running flag. Recalculates all active quantization grids (IOT, grain size, delay) using new BPM.

In playhead Mode 3: sets clock_bang_received flag for playhead advance.

clockstop Clears clock_running flag.

Preserves last calculated BPM and quantization grids for continued use. Posts current BPM value.

IOT Quantization

Snaps interonset time to rhythmic grid.

timesig <symbol> Set time signature for IOT quantization.

Format: "numerator/denominator" (e.g., "7/5", "4/4") Requires symbol argument. Parses using sscanf().

Validation: numerator > 0, denominator > 0

Posts error if invalid format or values.

Stores: time_sig_numerator, time_sig_denominator

Default: 4/4

quantize <int> Set IOT quantization note subdivision. Valid values: 1, 2, 4, 8, 16, 32, 64, 128

Represents note division (16 = 1/16 note)

Stores quant_note. Recalculates quant_grid_ms if BPM > 0:

Grid calculation:

$\text{ms_per_whole_note} = (60000.0 / \text{bpm}) \times 4.0$

$\text{quant_grid_ms} = \text{ms_per_whole_note} / \text{quant_note}$

Posts grid size in milliseconds if BPM available, otherwise defers calculation.

Default: 16 (1/16 note)

quant <float> Set IOT quantization amount (blend).

Range: 0.0 to 1.0

0.0 = no quantization (free timing)

1.0 = full quantization (locked to grid)

0.5 = 50% blend

Applied in perform cycle:

$\text{grain_trigger_period} = \text{base_period} \times (1 - \text{quant_amount}) + \text{quant_period} \times \text{quant_amount}$

Requires: quant_amount > 0, bpm > 1.0, quant_grid_ms > 0

Default: 0.0 (no quantization)

Grain Size Quantization

Snaps grain duration to rhythmic grid.

gs_timesig <symbol> Set time signature for grain size quantization.

Format: "numerator/denominator"

Validation and parsing identical to timesig.

Stores: gs_time_sig_numerator, gs_time_sig_denominator

Default: 4/4

gs_quantize <int> Set grain size quantization notesubdivision. Valid values: 1, 2, 4, 8, 16, 32, 64, 128

Calculation identical to quantize.

Stores gs_quant_note. Recalculates gs_quant_grid_ms if BPM > 0.

Default: 16 (1/16 note)

gs_quant <float> Set grain size quantization amount.

Range: 0.0 to 1.0

Blend between free grain_size and quantized grid.

Applied in perform cycle:

$\text{quant_grain_size_sec} = \text{gs_quant_grid_ms} / 1000.0$

$\text{quantized_grain_size} = \text{grain_size} \times (1 - \text{gs_quant_amount}) + \text{quant_grain_size_sec} \times \text{gs_quant_amount}$

Result assigned to scheduler->grain_size.

Requires: gs_quant_amount > 0, bpm > 1.0, gs_quant_grid_ms > 0

Default: 0.0 (no quantization)

Delay Time Quantization

Snaps grain delay time to rhythmic grid.

delay_timesig <symbol> Set time signature for delayquantization.

Format: "numerator/denominator"

Validation and parsing identical to timesig.

Stores: delay_time_sig_numerator, delay_time_sig_denominator

Default: 4/4

delay_quantize <int> Set delay quantization note subdivision. Valid values: 1, 2, 4, 8, 16, 32, 64, 128

Calculation identical to quantize.

Stores delay_quant_note. Recalculates delay_quant_grid_ms if BPM > 0.

Default: 16 (1/16 note)

delay_quant <float> Set delay quantization amount.

Range: 0.0 to 1.0

Blend between free delay_time and quantized grid.

Applied in perform cycle:

$\text{quant_delay_sec} = \text{delay_quant_grid_ms} / 1000.0$

$\text{quantized_delay} = \text{current_delay} \times (1 - \text{delay_quant_amount}) + \text{quant_delay_sec} \times \text{delay_quant_amount}$

Result assigned to grain_delay->delay_time.

Requires: delay_quant_amount > 0, bpm > 1.0, delay_quant_grid_ms > 0

Default: 0.0 (no quantization)

PITCH & SPEED

pitch_mode 0|1|2|3|4 Five pitch modes control relationship between speed parameter and playback rate.

Mode 0 - OFF (default)

speed <float>

Speed parameter = playback rate directly.

`speed_range <min> <max>` active if enabled.

Formula: $\text{final_speed} = \text{sample_param_range}(\text{speed_range}, \text{speed_inlet})$

Mode 1 - SEMITONES

Speed parameter = base pitch.

Fixed semitone transposition applied.

`speed_range` bypassed.

Formula: $\text{final_speed} = \text{speed} * 2^{(\text{semitones}/12)}$

`pitch_semitones <-24 to 24>`

Example: `pitch_semitones 7` = perfect fifth up

Mode 2 - RANGE

Speed parameter = base pitch.

Random semitone shift from range.

`speed_range` bypassed.

Formula: $\text{final_speed} = \text{speed} * 2^{(\text{random_semitones}/12)}$

Messages, see PARAMETER RANGES & MODULATION for generators:

`pitch_range <min> <max>`

`pitch_rand_type <generator>`

Example:

`pitch_range -12 12 (±1 octave)`

Mode 3 - SCALE

Speed parameter = base pitch.

Random note from scale/list of semitones (quantized pitch).

`speed_range` bypassed.

Formula: $\text{final_speed} = \text{speed} * 2^{(\text{scale_note}/12)}$

`pitch_scale <note1> <note2> ... <noteN>`

Example:

`pitch_scale 0 2 4 5 7 9 11` (major scale)

Up to 128 notes per scale.

Mode 4 - MIDI

Speed parameter = base pitch (assumed C4 = 60).

MIDI note input transposes from base.

speed_range bypassed.

Formula: $\text{final_speed} = \text{speed} * 2^{((\text{midi_note} - 60)/12)}$

MIDI input via inlet 19 (1-127, 0 = inactive)

Final speed clamped: -4.0 to 4.0 (± 48 semitones / 4 octaves)

MIDI mode "latches" to the last valid note until pitch_mode changes. Switching back to pitch_mode 4 will resume with this latched note.

DELAY

DD-4 style delay applied to grain output after Moog filter. Circular buffer with feedback loop and tone control.

Parameters

`gdelay_time <float>` Set delay time.

Range: 0.0 to 9.5 seconds

Clamped to bounds.

Smoothed internally to prevent clicks during time changes. Smoothing coefficient: 0.001 (approximately 20ms transition at 48kHz).

Signal inlet 10: updates if value in range (0.0, 10.0], else keeps current value.

`gdelay_feed <float>` Set feedback amount.

Range: 0.0 to 1.0

0.0 = single echo

1.0 = infinite regeneration

Clamped to bounds.

Feedback applied after tone filter in feedback loop.

Signal inlet 11: updates if value in range (0.0, 1.0], else keeps current value.

gdelay_tone <float> Set tone character via one-pole IIR low-pass filter in feedback loop.

Range: 0.0 to 1.0

0.0 = dark, heavily filtered (analog-style decay)

1.0 = bright, no filtering (digital-style)

Clamped to bounds.

Filter coefficient applied directly: $\text{filtered} = \text{tone} \times \text{delayed} + (1 - \text{tone}) \times \text{lpf_state}$

Higher values pass more high frequencies. Lower values create darker, warmer repeats.

Signal inlet 12: updates if value in range (0.0, 1.0], else keeps current value.

gdelay_mix <float> Set dry/wet mix.

Range: 0.0 to 1.0

0.0 = 100% dry (grain output only, no delay)

1.0 = 100% wet (delayed signal only)

Clamped to bounds.

Output calculation: $\text{output} = \text{dry} \times (1 - \text{mix}) + \text{wet} \times \text{mix}$

Signal inlet 13: updates if value in range (0.0, 1.0], else keeps current value.

gdelay_clear Clear delay buffers and filterstate.

Zeros `buffer_left`, `buffer_right`, `lpf_state_left`, `lpf_state_right`.

Immediate silence in feedback loop.

DISTORTION

Multi-mode waveshaping with pre/post filters and oversampling. Two position modes: per-grain (experimental) or post-mix (default). In post-mix mode, distortion is applied only AFTER recording as a monitoring effect.

Control

distortion_enable 0|1 Enable/disable distortion processing.

0 = bypass (passthrough)

1 = enabled

Signal inlet 14: updates if value in range (0.0, 1.0], else keeps current value.

distortion <float> Set distortion drive amount.

Range: 0.0 to 1.0

Maps to current_drive via lookup curve.

0.0 = clean (minimal drive)

1.0 = saturated (maximum drive)

distortion_position 0|1 Set distortion placement in signal chain.

Mode 0: Per-grain (experimental)

Applied to each grain sample during synthesis, before envelope. No oversampling. Uses grain_distortion_process_sample().

Signal chain: read → distort → envelope → pan → mix

Distortion IS recorded in this mode (affects grain output before recording).

Mode 1: Post-mix (default)

Applied to final output AFTER all recording operations.

Uses oversampling. Uses grain_distortion_process_block().

Signal chain: grains → delay → [RECORDING] → output → Moog → distortion → dac~

Distortion NOT recorded (monitoring/output only).

This placement prevents distortion artifacts from accumulating in overdub recordings.

Default: 1 (post-mix)

distortion_oversample_factor 1|2|4|8 Set oversampling multiplier for post-mix mode. Valid values: 1, 2, 4, 8 ?

Posts error if invalid value provided.

Higher values reduce aliasing at cost of CPU.

4x provides good quality/performance balance.

Per-grain mode ignores this setting (no oversampling).

Default: 4

Waveshaper Modes

dist_waveshaper_mode 0|1|2|3|4 Select waveshaping algorithm.

Mode 0: tanh (default)

Pure hyperbolic tangent.

Aggressive odd harmonics. Warm, tube-like saturation.

Mode 1: arctan

Pure arctangent.

Smooth odd harmonics. Gentler than tanh.

Mode 2: asymmetric

Different drive for positive/negative cycles.

Generates even harmonics.

Uses drive_pos and drive_neg parameters.

Mode 3: blend

Morph between tanh and arctan.

Uses dist_curve_blend parameter (0.0 = tanh, 1.0 = arctan).

Mode 4: polynomial

User-defined polynomial: $\text{output} = c1 \times x + c2 \times x^2 + c3 \times x^3$

Uses poly_c1, poly_c2, poly_c3 coefficients.

Precise harmonic control.

Default: 0 (tanh)

dist_curve_blend <float> Blend amount for Mode 3.

Range: 0.0 to 1.0

0.0 = full tanh (aggressive)

1.0 = full arctan (smooth)

dist_drive_pos <float> Positive cycle drive for Mode2. Range: 1.0 to 20.0

Higher values = more saturation on positive peaks.

dist_drive_neg <float> Negative cycle drive for Mode2.

Range: 1.0 to 20.0

Higher values = more saturation on negative peaks.

dist_poly_c1 <float> Linear coefficient for Mode 4.

Range: -10.0 to 10.0

Controls fundamental and odd harmonics.

Default: 1.0 (linear passthrough)

`dist_poly_c2 <float>` Quadratic coefficient for Mode 4.

Range: -10.0 to 10.0

Controls even harmonics (asymmetry).

`dist_poly_c3 <float>` Cubic coefficient for Mode 4.

Range: -10.0 to 10.0

Controls additional odd harmonics.

Pre-emphasis / De-emphasis (Inner Filters)

Frequency-dependent pre-distortion boost with matched post-distortion cut. Affects harmonic content without changing overall tonal balance.

`dist_emphasis_mode 0|1` Set pre-emphasis filter type.

0 = HP (high-pass): boost highs before distortion → brighter saturation

1 = LP (low-pass): boost lows before distortion → darker saturation

Argument ≥ 0.5 selects LP, < 0.5 selects HP.

Default: 0 (HP mode)

`dist_emphasis_freq <float>` Set emphasis corner frequency. Range: 100 to 5000 Hz

Higher freq = more extreme emphasis effect.

`dist_pregain <float>` Set pre-distortion gain stage.

Range: 0.1 to 10.0

Applied before emphasis filters and waveshaper.

1.0 = unity 1.0 = drive harder

< 1.0 = soften

Outer Filters

Applied outside emphasis/waveshaper core. Shape frequency response without affecting emphasis balance.

`distortion_pre_hp_freq <float>` Pre-tanh highpass frequency.

Range: 30 to 500 Hz

1-pole filter (6dB/octave).

Removes DC and low mud before saturation.

`distortion_pre_hp_mix <float>` Pre-tanh highpass dry/wet.

Range: 0.0 to 1.0

0.0 = full passthrough (no filtering)

1.0 = full filtered

`distortion_post_lp_freq <float>` Post-tanh lowpassfrequency.

Range: 2400 to 10000 Hz

2-pole Butterworth (12dB/octave).

Tames high-frequency harmonics after saturation.

`distortion_post_lp_mix <float>` Post-tanh lowpass dry/wet.

Range: 0.0 to 1.0

0.0 = full bright (no filtering)

1.0 = full dark

`distortion_notch_freq <float>` Notch filter centerfrequency.

Range: 20 Hz to (sample_rate/2 - 100) Hz

Bandstop filter for targeted frequency removal.

`distortion_notch_bw <float>` Notch filter bandwidth.

Range: 10 to 5000 Hz

Narrower = more surgical cut.

`distortion_notch_mix <float>` Notch filter dry/wet.

Range: 0.0 to 1.0

0.0 = inactive (no notching)

1.0 = full notch applied

Default: 0.0 (inactive)

MOOG LADDER FILTER

Classic 4-pole (24dB/octave) Moog ladder lowpass filter. Four cascaded one-pole stages with resonance feedback. Filter is applied only AFTER recording as a monitoring effect.

Parameters

`moog_cutoff <float>` Set filter cutoff frequency.

Range: 20 to 20000 Hz

Clamped to Nyquist frequency ($\text{sample_rate} / 2$).

Formula: $f = 2 \times \sin(\pi \times f_c / f_s)$

Cutoff coefficient clamped to [0.0, 1.0] for stability.

Signal inlet 15: updates if value in range [20.0, 20000.0], else keeps current value.

moog_resonance <float> Set resonance feedback amount.

Range: 0.0 to 4.0

0.0 = no resonance (flat lowpass)

3.5+ = self-oscillation (pitched sine tone at cutoff frequency)

4.0 = maximum feedback

Feedback formula: $fb = \text{resonance} \times (1.0 + 0.5 \times f^5)$

Nonlinear compensation prevents resonance drop at high cutoff frequencies.

Signal inlet 16: updates if value in range [0.0, 4.0], else keeps current value.

moog_mix <float> Set dry/wet mix.

Range: 0.0 to 1.0

0.0 = 100% dry (bypass, no filtering)

1.0 = 100% wet (fully filtered)

Output: $\text{output} = \text{dry} \times (1 - \text{mix}) + \text{wet} \times \text{mix}$

Signal inlet 17: updates if value in range [0.0, 1.0], else keeps current value.

moog_enable 0|1 Enable/disable filter processing.

0 = bypass (passthrough)

1 = enabled

When disabled or mix = 0.0, filter bypasses without processing.

PARAMETER RANGES & MODULATION

Per-grain parameter variation system. Each synthesis parameter can vary within a specified range using five modulation sources. Supports 21 modulatable parameters across grain synthesis, effects, and distortion.

Modulation Sources

Five generator types, each with 4 independent instances (1-4):

- **RAND_TYPE_RAND**

Basic pseudo-random. Seeded per instance. Uniform distribution [0.0, 1.0]. Fast, uncorrelated white noise variation.

- **RAND_TYPE_PERLIN_1D**

1D Perlin noise. Smooth continuous modulation. Coordinate advances by $\text{iot} \times \text{noise_frequency_scale}$ per grain. Decorrelated via instance offsets: [0, 1000, 2003, 3001]. Returns normalized [0.0, 1.0].

- **RAND_TYPE_PERLIN_2D**

2D Perlin noise. X-coordinate advances per grain, Y-coordinate = base parameter value.

Coupling between parameter value and modulation depth. Decorrelated via instance offsets: [0, 5000, 10007, 15013]. Returns normalized [0.0, 1.0].

- **RAND_TYPE_LORENZ**

Lorenz attractor. Bounded chaotic motion. Evolves via update iterations: $\text{int}(\text{iot} \times 100 \times \text{noise_frequency_scale})$. Capped 1-50 iterations per grain for stability. Output axis rotates per instance: 0=X, 1=Y, 2=Z (mod 3). Different initial positions per instance for decorrelation. Returns normalized [0.0, 1.0].

- **RAND_TYPE_NBODY**

N-body gravitational simulation (3-body problem). Bounded chaotic orbital motion.

Evolves via update iterations like Lorenz. Output mode selectable per instance (0-10):

Body positions, velocities, distances, angular momentum, total energy. Returns normalized [0.0, 1.0].

Modulatable Parameters Include:

- **Grain synthesis (sampled per grain trigger):**

speed, grainsize, grainstart, amplitude, pan, maxgrains

- **Timing (sampled per DSP block):**

iot (interonset time)

- **Effects (sampled per DSP block):**

Gdelay, gdelay_tone, moog_cutoff, moog_resonance, moog_mix

- **Distortion:**

distortion (intensity), dist_emphasis_freq, dist_pregain, dist_curve_blend, dist_drive_pos, dist_drive_neg, dist_poly_c1, dist_poly_c2, dist_poly_c3

Not modulatable:

scanrate (no range implemented)

Messages

- **param_range <param_name> <min> [max]** Set parameter modulation range.
 - Two-argument form (single value): Disables range, sets fixed value.
Example: param_range speed 1.0 → speed fixed at 1.0
 - Three-argument form (range): Enables modulation between min and max.
Example: param_range speed 0.5 2.0 → speed varies 0.5-2.0
- **rand_type <type_instance> [param_name]** Set modulation source for parameter(s).
 - type_instance format: {type}_{N} where N = 1-4
 - Types: rand_1...rand_4, perlin_1d_1...perlin_1d_4, perlin_2d_1...perlin_2d_4, lorenz_1...lorenz_4, nbody_1...nbody_4
 - One-argument form: Sets all parameters to specified source. *Example:*
rand_type perlin_1d_2
 - Two-argument form: Sets single parameter to specified source. *Example:*
rand_type lorenz_3 amplitude
- **noise_freq <float>** Set frequency scale for all 4 modulation instances.
 - Range: > 0.0 (1.0 = normal speed)
 - Affects Perlin coordinate advancement and Lorenz/N-body iteration count.
- **noise_freq_1 <float>**
- **noise_freq_2 <float>**
- **noise_freq_3 <float>**
- **noise_freq_4 <float>**

Set frequency scale for specific instance (1-4). Independent control of each modulation instance speed.

Lock

Sends current modulated values on outlet 9 then stops modulation.

param_lock <param1> <param2> <param3> ...

Example:

param_range pan 0.2 0.8

`rand_type perlin_1d_1 pan`

`param_lock pan` → "ligase~: pan locked at 0.547"

`param_lock pan amplitude grainsize` → locks all three at current values

Disables range modulation only.

Returns to inlet or last message value.

Without feedback routing:

→ Just stops modulation

→ Returns to inlet/message control

→ Values go to outlet 9 but aren't applied

With feedback routing (outlet 9 → inlet 1):

→ Stops modulation

→ Captures and applies current modulated values

→ "Locks" parameters at modulated state

N-Body Control

N-body instances have additional physics parameters (1-4):

- `nbody_epsilon <instance> <value>` Set softening parameter (0.001-0.1). Prevents singularities during close approaches.
- `nbody_damping <instance> <value>` Set velocity damping (0.0-0.1). Higher values = faster energy loss, more stable orbits.
- `nbody_pump <instance> <amount> <interval>` Set energy pump interval (velocity increment, iteration interval). Adds velocity periodically to sustain motion
- `nbody_G <instance> <value>` Set gravitational constant(0.1-10.0).
- `nbody_mode <instance> <mode>` Set output extraction mode (0-10).

11 output modes per instance:

- 0: Body 0 X position (instance 1 default)
- 1: Body 1 Y position (instance 2 default)
- 2: Body 2 X position (instance 3 default)
- 3: Distance body 0-1 (instance 4 default)
- 4: Velocity magnitude body 0
- 5: Velocity magnitude body 1
- 6: Velocity magnitude body 2
- 7: Distance body 0-2
- 8: Distance body 1-2
- 9: Angular momentum magnitude
- 10: Total system energy

Auto-normalizing bounds track min/max values.

Sampling Timing

- **Per grain trigger:** speed, grainsize, grainstart, amplitude,pan, and maxgrains
- **Per DSP block (smooth variation):** iot, gdelay, moog_cutoff,moog_resonance, moog_mix.

Modulation Mapping

Output value: $\text{min} + \text{random_value} \times (\text{max} - \text{min})$. Linear mapping using normalized random_value [0.0, 1.0] from the generator.

Generator Outlet Sends

`Modout<N>_source <type> <instance>`

N: outlet number (1-4)

type: rand | perlin_1d | perlin_2d | lorenz | nbody

instance: 1-4

Set Range

`modout<N>_range <min> <max>`

Per-Outlet Base Value

Set PERLIN_2D Y-coordinate for parameter decorrelation.

`param_base_value <param> <value>`

Example:

`rand_type perlin_2d_1 modout1`

`rand_type perlin_2d_1 modout2`

`param_base_value modout1 0.2`

`param_base_value modout2 0.8`

Result:

modout1 and modout2 use same instance but different Y-coordinates, producing decorrelated patterns.

Notes:

Default: 0.5 for all parameters

Only affects PERLIN_2D (ignored for other generator types)

Enables spatial decorrelation without using multiple instances

Parameter Slewing

Apply exponential smoothing to modulation output.

`param_slew <param> <coefficient>`

Example:

`rand_type rand_1 modout1`

`param_range modout1 0 1`

`param_slew modout1 0.9` → Very smooth, slow tracking

`param_slew modout1 0.5` → Moderate smoothing

`param_slew modout1 0.0` → Instant (no smoothing)

Range:

0.0: Instant response (disabled)

0.5: Moderate smoothing

0.99: Very slow tracking

1.0: Frozen (no updates)

Notes:

Algorithm: $\text{smoothed} = \text{smoothed} \times \text{slew} + \text{new} \times (1 - \text{slew})$

Works with all generator types

Generator Reset

Reset Perlin or Lorenz generators to initial state.

`perlin_reset <instance>`

`lorenz_reset <instance>`

Example:

`perlin_reset 1` → Resets Perlin 1D/2D coordinates to 0

`lorenz_reset 2` → Resets Lorenz attractor to initial position

`nbody_reset 3` → Resets N-body system (existing feature)

Perlin Reset:

Resets `noise_1d_coord` and `noise_2d_coord_x` to 0.0

Useful for synchronizing modulation patterns

Lorenz Reset:

Returns x, y, z to initial values (x0, y0, z0)

Each instance has different starting position:

Instance 1: (0.1, 0.0, 0.0)

Instance 2: (5.0, 10.0, 20.0)

Instance 3: (-3.0, -5.0, 10.0)

Instance 4: (8.0, 15.0, 30.0)

N-body Reset:

Resets to hierarchical initial configuration

Each instance has different mass distribution

Notes:

Instance range: 1-4

Does not reset frequency scaling (`perlin_frequency`)

Does not reset N-body physics parameters (epsilon, damping, etc.)

Defaults

All parameter ranges initialized to: min: 0.0, max: 1.0, rand_type: RAND_TYPE_RAND (instance 1), enabled: 0 (disabled). All noise_frequency_scale defaults to 1.0.

STATE

Export and inspect ligase~ state via outlet 9.

Useful for creating presets or routing values to other instances:

`query <param>` → Output single parameter value

`get_params` → Output all 20 parameter values

Debugging and inspection:

`get_inlets` → Print inlet guide to console

`get_ranges` → Output active param_range settings

`get_generators` → Output active rand_type assignments

`get_state` → Output everything (params + ranges + generators)

`query bpm` → returns calculated bpm

get_inlets

Print inlet documentation to console (no outlet).

Output: Console only

Shows current values and message syntax

query

Query single parameter value.

`query <param>`

20 queryable parameters:

speed, grainsize, grainstart, organize, scanrate, sos, iot, maxgrains,
gdelay, gdelay_feed, gdelay_tone, gdelay_mix, distortion,
moog_cutoff, moog_resonance, moog_mix, midi, env_skew, amplitude, pan

Per-block parameters - store modulated values when param_range.enabled:

gdelay_time, gdelay_feedback, gdelay_tone, gdelay_mix
moog_cutoff, moog_resonance, moog_mix

organize, sos, env_skew, iot

Per-grain parameters - Sample once per block for query when modulation enabled:

speed, grainsize, grainstart, amplitude, pan, maxgrains, distortion

query <param> and **get_params** return:

Inlet/ last message value when modulation is disabled

Current modulated value when modulation is active

Locked value when param_lock has been applied

Unconnected inlets preserve stored values if headless mode is enabled

get_params

Export all 20 parameter values.

Fixed order (speed → grainsize → ... → pan)

get_ranges

Export active parameter range configurations.

Output format:

param_range <param> <min> <max>

Only outputs enabled ranges

Messages can be sent back to restore range configuration

get_generators

Export active generator assignments.

Output format:

rand_type <generator>_<instance> <param>

Generator types:

rand, perlin_1d, perlin_2d, lorenz, nbody

Instance numbers 1-4

Only outputs modout1-4 (scheduler generators not accessible)

Does NOT export perlin_frequency or nbody settings

get_state

Export complete state (all three subsystems).

Output order:

1. All parameter values (20 messages)
2. All enabled ranges (variable)
3. All generator assignments (variable)

Calls get_params + get_ranges + get_generators

All messages are routable lists

Limitations

No quantization state - timesig/quant settings not exported